OPERATING SYSTEMS – ASSIGNMENT 1

Name: S. Sharvesh **Register No:** 192525458

TASK 8 – Evaluate CPU States and Protection (3 Marks)

Evaluate user mode and kernel mode, privileged instructions and mode bit; interrupts, traps and timer interrupt; what happens if any program can disable interrupts; and how hardware stops one program reading another program's memory.

Alternative Concept and Complexity Analysis: Compare polling with interrupts for worst-case abnormal-reading detection delay, CPU time and power.

Evidence Expected: Reasoned explanation applied to patient monitors.

ANSWER

8.1 User mode and kernel mode

The CPU operates in at least two privilege levels. User mode restricts ordinary applications from executing privileged instructions or directly accessing protected hardware and kernel memory. Kernel mode allows the OS to manage memory, devices, interrupts and scheduling. A hardware mode/privilege bit records the current privilege level. System calls and interrupts transfer execution to trusted kernel code; return-from-trap restores user mode.

Privileged instructions include operations such as disabling interrupts, changing page-table control registers and configuring hardware devices.

An interrupt is an asynchronous hardware event, such as a timer or device signal. A trap/exception is a synchronous event caused by the current instruction, such as a system call or invalid memory access.

The timer interrupt returns control to the scheduler so that one process cannot monopolise the CPU indefinitely.

If an ordinary monitor application could disable interrupts, it could prevent the scheduler and device handlers from running. Alarm processing could be delayed, the OS could hang, and other safety tasks could be starved.

Memory protection uses virtual address spaces and page-table permissions. The MMU checks each memory access; one process cannot read another process’s pages unless the OS deliberately maps shared memory and grants permission.

8.2 Alternative: polling instead of interrupts

If a sensor is checked every T milliseconds, an abnormal value that appears just after a check may wait almost T milliseconds before detection. Worst-case detection delay is approximately T plus the execution time of the polling routine. Interrupt-driven detection is instead bounded by sensor/interrupt delivery latency, interrupt masking time and handler/scheduling latency. For a 10 ms alarm requirement, a polling interval close to 10 ms leaves no time for processing and alarm output; it is therefore unsuitable unless the complete worst-case path is proven below the deadline.

Polling repeatedly consumes CPU cycles even when no event occurs, wasting energy and potentially increasing heat. Interrupts allow the CPU to perform other work or enter a low-power state until an event occurs. Polling may still be appropriate for very fast, predictable sampling where the hardware or RTOS uses a dedicated high-priority loop, but it must be justified by measured worst-case timing.

QUESTION FROM THE PROVIDED ASSIGNMENT